

30 Days Roadmap to Crack Amazon SDE Interview

Sep 25, 2026 · @Hareesh Rajendran

Overview

This is the exact roadmap I used to prepare for my Amazon interview in 30 days. The same system got me an offer of around ₹46 lakh CTC. Take the roadmap, not the salary number.

Do not prepare for Amazon. Prepare for the specific role Amazon is hiring for. That is the base of this whole plan.

Which level are you preparing for

Level	Where to put extra weight	Daily time
SDE-1 / fresher	DSA patterns, clean code, edge cases, 8 stories	3 to 4 hours on weekdays, 5 to 6 hours on weekends
SDE-2	System design, ownership stories, resume deep dive, 10 stories	3 to 4 hours on weekdays, 5 to 6 hours on weekends

Every day in this plan has tasks. Tick them off as you go.

The 7 things Amazon tests

1. Whether you understand the role, by breaking the job description down
2. DSA patterns, not a pile of collected questions
3. Coding out loud as if the interviewer is sitting next to you
4. Your resume, known line by line
5. Leadership Principles, prepared as stories

6. Mock interviews and an error log
7. How Amazon evaluates you, which is the most important part of the whole roadmap

30 Day Plan

If your interview is exactly 30 days away, this is how I would split the time across 7 phases. The day by day tasks for each phase are below.

Phase	Days	Focus
1. Foundation	1 to 5	Understand the role, DSA basics, language revision, resume deep dive, start stories
2. Patterns	6 to 12	Pattern based coding, system design basics, story writing
3. Strengthen	13 to 18	Medium and hard problems, resume topics, one full system design
4. Mocks	19 to 23	Coding, system design and behavioural mocks, error log after each
5. Fix	24 to 26	Fix weaknesses only, redo wrong questions, rehearse stories
6. Simulation	27 to 29	Full interview simulations in interview conditions
7. Rest	30	Review, rehearse, sleep

Phase 1: Foundation (Days 1 to 5)

Goal: understand the role, rate yourself honestly, and know your resume cold.

Day	Focus	Tasks	Output
1	Understand the role	Break the job description into technical skills, problem solving and behavioural expectations; note how much weight goes to coding and DSA, language and CS fundamentals, system design, projects and Leadership Principles; rate yourself out of 10 in each	Self rating sheet and a 30 day time split
2	DSA basics and language	Revise arrays, strings, hashing and time and space complexity; revise your language's collections, OOP and concurrency basics; solve 2 easy problems out loud	2 problems solved with explanation
3	DSA basics continued	Linked lists, stacks, queues, recursion basics; write code with correct syntax instead of pseudocode; solve 2 problems	Language cheat sheet
4	Resume deep dive	For every bullet ask: from what to what, how did I measure it, what was the bottleneck, what did I personally change, why this solution, what were the alternatives; mark every bullet you cannot discuss comfortably for 5 minutes	Resume gap list
5	Behavioural start	List 8 to 10 story topics: failure, disagreement, difficult deadline, ownership, ambiguity, difficult technical decision, mistake, measurable achievement; write raw points for 3 stories	Story topic list and 3 rough stories

Checkpoint: by the end of Day 5 your weak areas should be clear. Adjust your time split to match.

Watch: Product Based Companies Rejected You? Here's Why for resume and DSA pattern mistakes, and What FAANG Engineers Wish They Knew as Freshers for the overall approach.

Phase 2: Patterns (Days 6 to 12)

Goal: pattern recognition, not 200 random problems. Solve 1 or 2 timed problems daily, 30 to 40 minutes each, out loud.

Day	Focus	Problems	Also do
6	Arrays and hashing	Two Sum, Group Anagrams, Top K Frequent Elements	For every problem answer 3 questions: what is the pattern, why does it work, what changes if constraints change
7	Two pointers and sliding window	3Sum, Container With Most Water, Longest Substring Without Repeating Characters, Minimum Window Substring	Edge case list: empty input, duplicates, largest constraint
8	Binary search	Binary Search, Search in Rotated Sorted Array, Koko Eating Bananas	System design basics: load balancer, L4 vs L7
9	Stacks and queues	Valid Parentheses, Daily Temperatures, Min Stack	System design basics: caching, SQL vs NoSQL, database indexing
10	Trees	Maximum Depth, Level Order Traversal, Validate BST, Lowest Common Ancestor	System design basics: capacity estimation (QPS, storage), API design
11	Graphs part 1	Number of Islands, Clone Graph, Course Schedule	Story writing: 4 stories in full STAR format
12	Review day	Redo the 3 problems you struggled with most; one self timed 45 minute mock	Finish the first draft of 6 stories

Checkpoint: on Day 12, given a new problem you should recognise the pattern within 2 minutes.

Watch: STOP watching Coding Videos for how to learn patterns, 3 Algorithms That Make Developers STANDOUT, Leetcode vs Projects Reality Check, Load Balancer Explained and What REALLY Happens When You Type google.com for system design basics.

Phase 3: Strengthen (Days 13 to 18)

Goal: medium and hard problems, resume topics, and one full system design.

Day	Focus	Problems or tasks	Also do
13	Heaps	Kth Largest Element, Merge k Sorted Lists, Find Median from Data Stream	Explain complexity out loud
14	Recursion and backtracking	Subsets, Permutations, Combination Sum, Word Search	Draw the recursion tree
15	Basic DP	Climbing Stairs, House Robber, Coin Change, Longest Increasing Subsequence	Write the state and transition
16	Graphs part 2	Word Ladder, Rotting Oranges, Pacific Atlantic Water Flow, Network Delay Time	BFS vs DFS vs Dijkstra decision rule
17	Hard mix and resume topics	2 timed problems, 45 minutes each	Revise Redis, Kafka, AWS, microservices and database topics from your resume five levels deep
18	Full system design	One system end to end (URL shortener or chat app): requirements, estimates, API, data model, scaling, trade offs	Write stories 7 to 10 and add numbers to every result

Checkpoint: by Day 18 you have 8 stories ready, no weak resume bullets, and can explain one system design in 45 minutes.

Watch: Microservices Explained: What Interviewers Actually Ask, Cloud, EC2, S3, RDS Explained and How Whatsapp Works: System Design for system design and cloud fundamentals.

Phase 4: Mocks (Days 19 to 23)

Goal: proper 45 to 60 minute mocks. Someone casually asking you two questions is not a mock. After every mock fill the error log: what mistake did I make, why did I make it, what will I change next time.

Day	Mock	What it tests	After the mock
19	Coding mock 1	Clarify, approach, code, test, complexity, optimise	Error log entry
20	System design mock 1	Requirements, scale estimates, trade offs, requirement changes	Error log entry
21	Behavioural mock 1	STAR structure, Action heavy answers, numbers in results	Error log entry and story fixes
22	Coding mock 2	The specific weakness from Day 19	Error log review
23	Mixed mock	One coding round plus design or behavioural, whichever is weakest	Top 5 weaknesses list

Checkpoint: by Day 23 you have a top 5 weaknesses list. That list is the plan for the next 3 days.

Watch: Crack 42+ LPA Job with 4 Months of Preparation and How I Cracked Siemens 15 LPA+ Package for real interview experiences.

Phase 5: Fix (Days 24 to 26)

Goal: do not learn new things. Fix only the weaknesses in your error log.

Day	Focus	Tasks
24	Redo mistakes	Re-solve every question you got wrong, fresh, with a timer
25	Weak DSA patterns	Take your top 2 weak patterns, solve 4 problems in each, explain the pattern out loud
26	Story rehearsal	Say every story out loud, record yourself, check that Action is about 60 percent of the answer, add numbers

Phase 6: Simulation (Days 27 to 29)

Goal: full practice in interview conditions. Timer on, no notes, camera on, thinking out loud.

Day	Simulation	Notes
27	Coding 45 min, system design 60 min, behavioural 45 min, back to back	Real interview rhythm with short breaks
28	Same format with different problems	Start with your weakest area
29	Lighter simulation, fix your last 3 errors	Answer the 20 questions below honestly

Phase 7: Day 30

Do not try to learn 20 new things. Review, rehearse and sleep.

- [ ] Re-read your error log
- [ ] Rehearse story headlines and Action points
- [ ] Quick check: can you explain every resume bullet for 5 minutes
- [ ] Review your complexity cheat sheet and language syntax cheat sheet
- [ ] Interview logistics ready: link, laptop, charger, internet, quiet room

- [ ] Sleep early

At this point your job is not to know more. It is to perform better on what you already know.

Watch: Interview Secrets from Students Who Landed Jobs at Deloitte, KPMG, Evri Holdings for how others prepared.

DSA Patterns

Knowing the pattern is half the job. The interviewer will not tell you this is a sliding window problem. You have to recognise it.

For each pattern, pick representative questions and check that you can answer three things: what is this pattern, why does it work, and what changes if the constraints change.

Pattern	Signal to recognise it	Representative problems
Arrays and hashing	Duplicates, frequency, O(1) lookup	Two Sum, Group Anagrams, Top K Frequent Elements
Two pointers	Sorted array, pair or triplet, in place	3Sum, Container With Most Water, Remove Duplicates
Sliding window	Longest or shortest subarray or substring with a condition	Longest Substring Without Repeating Characters, Minimum Window Substring
Binary search	Sorted or monotonic, minimise or maximise the answer	Search in Rotated Sorted Array, Koko Eating Bananas
Stacks and queues	Nesting, next greater element, order processing	Valid Parentheses, Daily Temperatures, Min Stack
Trees	Hierarchy, recursion on left and right, BFS by level	Validate BST, Level Order Traversal, Lowest Common Ancestor
Graphs	Connections, islands, dependencies, shortest path	Number of Islands, Course Schedule, Word Ladder
Heaps	Top K, running median, merging sorted streams	Kth Largest Element, Merge k Sorted Lists, Find Median from Data Stream
Recursion and backtracking	Generate all combinations, choices with undo	Subsets, Permutations, Combination Sum, Word Search
Basic DP	Overlapping subproblems, count ways, min or max cost	Climbing Stairs, House Robber, Coin Change

Follow this sequence on every problem:

1. Clarify the problem
2. Explain your approach
3. Write the code with correct syntax
4. Test it
5. Analyse complexity
6. Optimise

Do not stop once the code works. Ask: what if the input is empty, what if there are duplicates, what happens at the largest constraint, can the complexity be improved.

Watch: Vanakkam DSA Live Cohort: How to choose a course and actually learn DSA and Your Competition Is Using This DSA Method.

Leadership Principles Stories

Do not memorise definitions. Build 8 to 10 stories, where each story covers 2 or 3 Leadership Principles.

Story	Situation prompt	Likely Leadership Principles	Numbers to add
1. Failure	A project or decision that failed	Ownership, Learn and Be Curious	What broke, the cost, what you changed after
2. Disagreement	A technical disagreement with your manager or a teammate	Have Backbone; Disagree and Commit, Earn Trust	Impact of the decision
3. Difficult deadline	Tight time, large scope	Deliver Results, Bias for Action	Days saved, scope trimmed
4. Ownership	A time you took ownership of something outside your job	Ownership, Customer Obsession	Users or teams affected
5. Ambiguity	A situation where requirements were unclear	Are Right A Lot, Invent and Simplify	Decisions made without full information
6. Technical decision	A difficult choice between two or more options	Dive Deep, Insist on the Highest Standards	Latency, cost, throughput comparison
7. Mistake	A mistake you made and how you fixed it	Earn Trust, Ownership	Time to detect and fix
8. Measurable achievement	A success with clear numbers	Deliver Results, Think Big	X to Y, X users, X hours saved
9. Helping others (optional)	A time you helped a junior or teammate grow	Hire and Develop the Best	People helped, outcome
10. Simplifying (optional)	A time you simplified a complex process	Invent and Simplify, Frugality	Steps or cost reduced

Structure: STAR

Situation. Task. Action. Result.

- Most of the answer should be Action. What did you do, not what did the team do.
- Keep Situation and Task to about 30 seconds. Do not spend 4 minutes on context.
- Put numbers in the Result: latency reduced from X to Y, handled X users, saved X hours.
- If you start thinking of a story while the interviewer waits, you are not prepared.

Story writing template

Fill this in for every story:

```
Story name:
Situation (2 lines):
Task (1 line):
Action (5 to 6 points, use I not we):
Result (with numbers):
Leadership Principles covered:
Follow up questions expected:
```

The 20 Questions You Must Answer

Before your interview, sit down and answer these 20 questions honestly. If any answer is not a clear yes, use Days 24 to 26 to fix it.

No	Question	If the answer is No
1	Can I solve a completely new coding problem while explaining my thinking from start to finish?	Not a memorised question, a genuinely new one. Do timed problems out loud every day
2	Can I recognise common DSA patterns without being told which pattern to use?	Revise the signal column in the pattern table
3	Can I explain why my chosen approach works, not just write the code?	After every solution, write two lines on why it works
4	Can I explain the time and space complexity of my solution clearly?	State the complexity after every problem
5	Can I handle edge cases and changing constraints without panicking?	Use the empty input, duplicates, largest constraint checklist
6	Can I write syntactically correct code without depending on pseudocode?	Practise writing code in a plain notepad without an editor
7	Can I defend every technical statement on my resume if the interviewer goes five levels deeper?	If not, go back to your resume. Repeat the Day 4 sheet
8	Can I explain exactly what I personally contributed to every project?	Say I, not we
9	Can I explain why I chose the technologies and solutions mentioned in my resume?	Write the alternatives and trade offs for each technology
10	Do I have at least five strong stories around ownership, failure, conflict, ambiguity and measurable impact?	Complete the story bank. If you start thinking of a story while the interviewer waits, you are not prepared
11	Can I structure my behavioural answers using Situation, Task, Action and Result?	Fill in the STAR template
12	Can I clearly explain what I personally did in every behavioural story?	Action should be about 60 percent of the answer
13	Do my stories have numbers, metrics or measurable outcomes wherever possible?	Add X to Y to every Result
14	Can I design a system, estimate its scale and explain my trade offs without copying a memorised diagram?	If not, do more system design practice
15	Can I explain why I chose one system design approach over another?	Write the alternative and the reason for every decision
16	Can I handle a system design requirement changing in the middle of the interview?	Ask a friend to change the requirements during a mock
17	Have I completed serious mock interviews where someone evaluates how I communicate and think?	Do at least 3 serious mocks
18	Do I have an error log showing the mistakes I repeatedly make during preparation?	Start one now, template is below
19	Have I fixed the weaknesses identified from my mock interviews instead of just collecting scores?	Repeat Phase 5
20	When I don't know the answer, do I know how to think my way towards it?	Clarify, break the problem down, test one assumption, keep moving

Question 20 is the most important one. You will get stuck. Everyone does. What matters is what happens after you get stuck.

Templates

Error log

Fill this in after every mock. Plan your next mock to test that exact weakness. Three serious mocks are better than ten random ones.

Date	Mock type	What mistake did I make	Why did I make it	What will I change next time
Day 19	Coding	Started coding without clarifying the problem	Rushed under time pressure	Spend the first 2 minutes on clarifying questions only
Day 20	System design	No scale estimates	Not confident with numbers	Make QPS and storage estimates the first step

The two rows above are examples. Replace them with your own.

Mock interview scorecard

Whoever runs your mock can fill this in. Score each row from 1 to 5.

Area	What to look for	Score
Clarify	Do they ask about requirements and constraints	
Structure	Do they explain the approach before coding	
Code	Correct, clean, correct syntax	
Testing	Edge cases and a dry run	
Complexity	Time and space stated correctly	
Communication	Thinking out loud, clear explanation	
Recovery	How they move forward after getting stuck	

Resume deep dive sheet

Answer these for every bullet. If you cannot discuss a bullet comfortably for 5 minutes, mark it.

Bullet	From what to what	How I measured it	Bottleneck	What I personally changed	Why this solution	Alternatives
Improved API performance by 40 percent						

Anything on your resume can become an interview question. Whether it is Redis, Kafka, AWS, microservices or databases, assume the interviewer can take you five levels deeper.

Number 7: Prepare for How Amazon Evaluates You

This is the most important part of the roadmap. Stop asking what questions Amazon will ask. Start asking what they are actually trying to evaluate.

Round	What Amazon is actually checking
Coding	Can you clarify the problem, structure your thinking, communicate, write correct code, test it and improve it
System design	Practicality, reliability, efficiency, optimisation and scalability. They will not just check whether you memorised one architecture
Behavioural	What happened, what did you do, why did you make that decision, and what was the result

Why this matters:

- You can solve 300 coding questions and still struggle with a new one
- You can memorise every Leadership Principle and still give a weak behavioural answer
- You can watch 50 system design videos and still freeze when one requirement changes

The interview tests how you think when the answer is not in front of you.

Daily drills for this

1. When you see a problem, spend 2 minutes on clarifying questions only, no code
2. After every solution, say in one line why it works
3. After every solution, change one constraint (input 10 times bigger, memory limit, streaming data) and adjust
4. When stuck: clarify, break the problem down, test one assumption, keep moving
5. For behavioural answers: Action, reason for the decision, result. Do not recite definitions

Watch: Don't Learn Coding Until You Watch This and ChatGPT Is Way Scarier Than You Think on why independent thinking matters.

Related Videos

Videos from my channel that match each part of the roadmap.

Roadmap part	Video	When to watch
Role and resume	Product Based Companies Rejected You? Here's Why	Day 1 to 5
Overall approach	What FAANG Engineers Wish They Knew as Freshers	Day 1 to 5
DSA patterns	STOP watching Coding Videos	Day 6 to 12
DSA patterns	3 Algorithms That Make Developers STANDOUT	Day 6 to 12
DSA practice	Leetcode vs Projects Reality Check	Day 6 to 12
DSA learning method	Your Competition Is Using This DSA Method	Day 6 to 12
DSA learning method	How to choose a course and actually learn DSA	Day 6 to 12
System design basics	Load Balancer Explained	Day 8
System design basics	What REALLY Happens When You Type google.com	Day 10
System design practice	How Whatsapp Works: System Design	Day 18
Resume tech topics	Microservices Explained: What Interviewers Actually Ask	Day 17
Resume tech topics	Cloud, EC2, S3, RDS Explained	Day 17
Interview experiences	Crack 42+ LPA Job with 4 Months of Preparation	Day 19 to 23
Interview experiences	How I Cracked Siemens 15 LPA+ Package	Day 19 to 23
Interview experiences	Interview Secrets from Students Who Landed Jobs	Day 30

Roadmap part	Video	When to watch
Thinking under pressure	Don't Learn Coding Until You Watch This	Day 24 to 26
Thinking under pressure	ChatGPT Is Way Scarier Than You Think	Day 24 to 26

Vanakkam DSA Cohort 7

If you want to build your DSA foundation properly, Vanakkam DSA Cohort 7 starts on Oct 1. I have pinned the registration link in the comments.

- 1,000+ learners have enrolled in the program
- 200+ DSA problems taught systematically, from fundamentals to advanced problem solving
- Built to develop pattern recognition instead of answer memorisation
- Mentorship and doubt solving
- Regular check-ins on where you are, what you have completed and what to focus on next
- A structured six month DSA journey

Watch: Vanakkam DSA Cohort 7 Is HERE

Last word

The goal is not to walk into Amazon hoping you get a question you already know. The goal is to walk in knowing how to think when you get a question you don't.

Hareesh Rajendran Founder, Vanakkam DSA